

# Reviewer Pack — Hypercontractive Term-Support Stable Rank Bounds Monotone Cl...

Entry fad53ef695ab · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 20:14:23 UTC

## Hypercontractive Term-Support Stable Rank Bounds Monotone Clique DNF

Entry ID: fad53ef695ab **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 20:14:23 UTC

### 1. Conjecture

**Field A** (mathematical branch): Fourier analysis / hypercontractivity — Bonami-Beckner noise-operator Gram kernels on DNF-term SUPPORT sets, evaluated through the stable rank  $\sigma(K) = (\sum \lambda_i)^2 / \sum \lambda_i^2$  of the resulting term-correlation matrix, in the Kahn-Kalai-Linial / Friedgut / Mossel tradition but used as a SYNTACTIC DNF invariant (a property of the formula, not its truth table) — an angle with essentially no presence in monotone-circuit lower-bound literature outside Razborov-approximator analogies **Field B** (complexity object): Monotone DNF / formula complexity for the k-CLIQUE indicator on v-vertex graphs (Razborov 1985 regime, target measure  $\mu$  on monotone DNFs satisfying (i) submodular under conjunction, (ii)  $O(\log n)$  for poly-size DNFs, (iii)  $\Omega(n)/\Omega(v)$  for the k-CLIQUE indicator), accessed through the canonical minterm DNF where each term is an edge-set of a k-clique

#### Statement:

For a monotone DNF  $F = \bigvee_{i=1}^s T_i$  on  $N$  variables with each term  $T_i = \bigwedge_{k \in S_i} x_k$  indexed by its support set  $S_i \subseteq [N]$ , define the Bonami-Beckner term-correlation Gram matrix  $K(F) \in \mathbb{R}^{s \times s}$  at noise parameter  $\rho=1/2$  by  $K(F)[i,j] := 2^{-\{|S_i|-|S_j|\}} \cdot (3/2)^{|\{S_i \cap S_j\}|}$ , and set  $\mu(F) := \log_2((\sum_i \lambda_i(K))^2 / \sum_i \lambda_i(K)^2)$ , the log of the stable rank of  $K(F)$ . Then (i) for every monotone DNF  $F$  on  $N$  variables with  $s$  terms,  $\mu(F) \leq 4 \cdot \log_2(2N+s)$ ; (ii) for the  $k$ -CLIQUE $_v$  indicator on  $N=C(v,2)$  edge variables with  $k=\lceil \log_2 v \rceil$ , in its canonical minterm DNF (one term per  $k$ -clique of  $K_v$ , support = edge set of the clique),  $\mu(F_{k\text{-CLIQUE}_v}) \geq v/4$ ; (iii)  $\mu$  is submodular under conjunction:  $\mu(F \wedge G) \leq \mu(F) + \mu(G)$  for any monotone DNFs  $F, G$  (writing  $F \wedge G$  in its distributed-out form  $\bigvee_{i,j} T_i \wedge T'_j$ ). A single instance violating (i), (ii), or (iii) refutes the conjecture.

#### Rationale (proposer's reasoning):

The Bonami-Beckner kernel at  $\rho=1/2$  admits a per-coordinate rank-2 factorization  $(f(a,b)=2^{\{-a-b\}(3/2)}\{ab\})$  with  $\det > 0$ , so  $K(F) = \bigotimes_k K_k$  restricted to the chosen supports — a hypercontractive Gram structure that controls effective dimensionality of the bottom monomial layer; poly-size DNFs cannot escape the trivial stable-rank  $\leq s \leq \text{poly}(N)$  bound, giving (i). The rigid intersection lattice of  $k$ -clique edge sets (two  $k$ -cliques sharing  $j$  vertices share  $C(j,2)$  edges, partitioning the  $s^2=C(v,k)^2$  pairs into  $k+1$  strongly-regular orbits) injects  $v$  many independent spectral components into  $K$ , driving stable rank to  $\geq 2^{\{v/4\}}$ . Submodularity under conjunction follows from tensor structure on disjoint variables (equality) and Cauchy-Schwarz contraction of merged Gram blocks on shared

variables; crucially  $\mu$  is a syntactic invariant of the DNF, dodging NATURAL\_PROOFS since computing it requires the formula, not the truth table.

**Taxonomy category:** FOURIER\_ANALYTIC (status at proposal time: alive)

## 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** b1a60d5281c48a3f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

Across 30 seeds for Family A ( $N \in \{20, 30, 40\}$ ), 30 relabelings for Family B ( $v \in \{6, 7, 8, 9\}$ ), and 30 seed-pairs for Family C ( $N=20, s \in \{6, 8\}$ ), every instance must satisfy (i)  $\mu(F) \leq 4 \cdot \log_2(2N+s)$ , (ii)  $\mu(F_{\{k\text{-CLIQUE}_v\}}) \geq v/4$ , and (iii)  $\mu(F \wedge G) \leq \mu(F) + \mu(G)$ , with numeric tolerance  $1e-9$ .

## 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.86       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.78       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.86       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - hypercontractivity noise operator stable rank monotone DNF lower bound - Bonami Beckner Gram matrix clique minterm Razborov approximation - term correlation kernel submodular DNF Fourier clique formula complexity

**Top relevant hits considered:** - [<http://arxiv.org/abs/1806.02734v3>] Spectral lower bounds for the orthogonal and projective ranks of a graph - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/1110.6594v1>] Around Operator Monotone Functions - [<http://arxiv.org/abs/2501.12063v2>] Gram-like matrix preserving extensions and completions of noncommutative polynomials - [<http://arxiv.org/abs/2510.18132v1>] Beurling Nyman Geometry and Gram Matrix Structure, Ladder Density and Polynomial Decay via Mellin Smoothing - [<http://arxiv.org/abs/2010.04691v4>] A graph theoretical framework for the strong Gram classification of non-negative unit forms of Dynkin type A - [<http://arxiv.org/abs/1208.2294v1>] Learning pseudo-Boolean k-DNF and Submodular Functions - [<http://arxiv.org/abs/2109.04525v2>] Sharper bounds on the Fourier concentration of DNFs

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90s$  wall time per cycle **Execution:** rc=1, elapsed=0.1s

## 5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0.0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_eigenvalues(A):
    n = len(A)
    if n > 20:
        raise ValueError("Matrix too large for this implementation")

    # Initialize a random matrix Q
    Q = [[random.random() for _ in range(n)] for _ in range(n)]

    for _ in range(100):
        # Compute  $Q^T * A * Q$ 
        QTA = matrix_multiply(matrix_transpose(Q), A)
        QTAQ = matrix_multiply(QTA, Q)

        # Compute eigenvalues (diagonal elements of QTAQ)
        eigenvalues = [QTAQ[i][i] for i in range(n)]

        # Update Q using QR decomposition (simplified)
        for i in range(n):
            norm = math.sqrt(sum(Q[i][j]**2 for j in range(n)))
            if norm > 0:
                for j in range(n):
                    Q[i][j] /= norm

    return eigenvalues

def compute_mu(K):
    eigenvalues = matrix_eigenvalues(K)
    mu_2 = sum(e**2 for e in eigenvalues)
    mu_1 = sum(e for e in eigenvalues)
    if mu_1 == 0:
        return 0.0
    return math.log2(mu_2 / mu_1)

def generate_random_dnf(N, s, seed):
    random.seed(seed)
```

```

terms = []
for _ in range(s):
    support_size = random.randint(3, 6)
    support = set(random.sample(range(N), support_size))
    terms.append(support)
return terms

def build_gram_matrix(F, rho=0.5):
    s = len(F)
    K = [[0.0 for _ in range(s)] for _ in range(s)]
    for i in range(s):
        for j in range(s):
            intersection = len(F[i] & F[j])
            K[i][j] = 2**(-len(F[i]) - len(F[j])) * (3/2)**intersection
    return K

def generate_clique_dnf(v, k, seed):
    random.seed(seed)
    vertices = list(range(v))
    random.shuffle(vertices)
    cliques = list(itertools.combinations(vertices, k))
    terms = []
    for clique in cliques:
        support = set()
        for i, j in itertools.combinations(clique, 2):
            support.add(i * v + j)
        terms.append(support)
    return terms

def dnf_conjunction(F, G):
    result = []
    for term_f in F:
        for term_g in G:
            result.append(term_f | term_g)
    return result

def run_trial(seed):
    random.seed(seed)
    N = random.choice([20, 30, 40])
    s = N
    F = generate_random_dnf(N, s, seed)
    K = build_gram_matrix(F)
    mu = compute_mu(K)
    bound = 4 * math.log2(2 * N + s)

    if mu > bound + 1e-9:
        return {
            "metric_name": "mu(F)",
            "metric_value": mu,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"mu(F) = {mu} > bound = {bound}"
        }

v = random.choice([6, 7, 8, 9])
k = math.ceil(math.log2(v))

```

```

F_clique = generate_clique_dnf(v, k, seed)
K_clique = build_gram_matrix(F_clique)
mu_clique = compute_mu(K_clique)
lower_bound = v / 4

if mu_clique < lower_bound - 1e-9:
    return {
        "metric_name": "mu(F_clique)",
        "metric_value": mu_clique,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"mu(F_clique) = {mu_clique} < lower_bound = {lower_bound}"
    }

F1 = generate_random_dnf(10, 6, seed)
G1 = generate_random_dnf(10, 6, seed + 1)
F_conj = dnf_conjunction(F1, G1)
K_conj = build_gram_matrix(F_conj)
mu_conj = compute_mu(K_conj)
K_F1 = build_gram_matrix(F1)
K_G1 = build_gram_matrix(G1)
mu_F1 = compute_mu(K_F1)
mu_G1 = compute_mu(K_G1)

if mu_conj > mu_F1 + mu_G1 + 1e-9:
    return {
        "metric_name": "mu(F_conj)",
        "metric_value": mu_conj,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"mu(F_conj) = {mu_conj} > mu(F1) + mu(G1) = {mu_F1 + mu_G1}"
    }

return {
    "metric_name": "mu(F)",
    "metric_value": mu,
    "instances_tested": 3,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_valid_trials")
        sys.exit(0)

    mean = sum(metric_values) / len(metric_values)

```

```

std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
else:
    failing_seeds = [r["seed"] for r in results if not r["conjecture_holds"]]
    counterexamples = [r["counterexample"] for r in results if not r["conjecture_holds"]]
    print(f"RESULT: FALSIFIED counterexample=\"{counterexamples[0]}\" first_failing_seed={failing_seeds[0]}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4d4e1b2.py", line 166, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4d4e1b2.py", line 107, in run_trial
    mu = compute_mu(K)
         ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4d4e1b2.py", line 56, in compute_mu
    eigenvalues = matrix_eigenvalues(K)
                  ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4d4e1b2.py", line 33, in matrix_eigenvalues
    raise ValueError("Matrix too large for this implementation")
ValueError: Matrix too large for this implementation

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with a ValueError ('Matrix too large for this implementation') before any instance could be evaluated, so no support\_fraction was produced and the pre-registered criterion is neither met nor falsified. | next: Replace the bespoke matrix\_eigenvalues routine with numpy.linalg.eigvalsh (or scipy) so the Gram matrices for Families A/B/C can be diagonalized at the required sizes, then rerun the 270+ instance battery.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider   | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max | opus             | 0         | 0   | 202476       |
| 2 | preregistration | claude_max | opus             | 0         | 0   | 7818         |
| 3 | novelty         | claude_max | opus             | 0         | 0   | 4773         |
| 4 | novelty         | claude_max | opus             | 0         | 0   | 9977         |
| 5 | test_gen        | mistral    | codestral-latest | 0         | 0   | 321583       |
| 6 | test_gen        | mistral    | codestral-latest | 0         | 0   | 314339       |
| 7 | test_gen        | mistral    | codestral-latest | 0         | 0   | 321660       |
| 8 | test_gen        | mistral    | codestral-latest | 0         | 0   | 313508       |
| 9 | judge           | claude_max | opus             | 0         | 0   | 6963         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1503097 ms total latency. Provider mix: {'claude\_max': 5, 'mistral': 4}

(full prompt+response transcripts available in *research/audit/fad53ef695ab.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/fad53ef695ab.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/fad53ef695ab.tar.gz* (if generated)